CS 61A — Week 2

Control, boolean expressions, iteration, higher-order functions, environments (Lectures 3–5, Composing Programs 1.4–1.6)

§1 Facts used from Week 1

(W1) Call expression: evaluate the operator, evaluate the operands left to right, apply the function.

(W2) Calling a user-defined function: new local frame, bind parameters to arguments, execute the body.

(W3) A name is looked up in the current frame first, then in later frames of the environment.

(W4) `print` displays its arguments and returns `None`. A function with no `return` returns `None`. The interpreter does not display `None`.

(W5) `n % 10` is the last digit of `n`; `n // 10` removes the last digit.

§2 This week

§2.1 Print versus return, once more (Lecture 3)

With `def f(x): return x + 1`, compare four versions of `h` called as `h(3)`:

body of `h(x)`	displays	returns	why
`f(x)`	nothing	`None`	the value `4` is computed and dropped; no `return`
`print(f(x))`	`4`	`None`	no `return`
`return print(f(x))`	`4`	`None`	returns the value of the `print` call, which is `None`
`return f(x)`	nothing	`4`	

Print, then return, the value of `f(x)`. Two candidates:

```
def h(x):                def h(x):
    print(f(x))              y = f(x)
    return f(x)              print(y)
                             return y
```

They differ when `f` is not pure. The left version calls `f` twice, so any side effect of `f` happens twice. With `f = print`: the left version of `h(2)` displays `2`, `None`, `2`; the right version displays `2`, `None`. **Call once, store the value in a name, use the name.**

§2.2 Conditional statements (Lecture 3, CP 1.5)

```
if x > 2:
    print('big')
elif x > 0:
    print('less big')
else:
    print('not pos')
```

Execution rule for a conditional statement.

1. Evaluate the header expression of the first clause.
2. If it is a true value, execute that clause's suite and skip all remaining clauses.
3. Otherwise go to the next clause. An `else` clause is executed if no earlier clause was.

- One `if`/`elif`/`else` statement executes *at most one* suite. Two separate `if` statements are unrelated: both can run. With `x = 10`, "`if x > 2: print('big')`" followed by a separate "`if x > 0: print('positive')`" displays both lines; with `elif` only `big`.
- **False values**: `False`, `0`, `None`, `''` (the empty string). Every other value seen so far is a true value, including negative numbers and nonempty strings.
- Comparison operators: `>`, `<`, `>=`, `<=`, `==`, `!=`. They evaluate to `True` or `False`. `=` is assignment; `==` is comparison.

§2.3 Boolean operators and short-circuiting (Lecture 4)

The question. `1 or 2` is not `True`; it is `1`. And `print(1) or 2` displays `1` and has value `2`. What rule produces these?

To evaluate `<left> and <right>`:

1. Evaluate `<left>`.
2. If the result is a false value `v`, the whole expression evaluates to `v`. `<right>` is not evaluated.
3. Otherwise the expression evaluates to the value of `<right>`.

To evaluate `<left> or <right>`:

1. Evaluate `<left>`.
2. If the result is a true value `v`, the whole expression evaluates to `v`. `<right>` is not evaluated.
3. Otherwise the expression evaluates to the value of `<right>`.

`not <exp>` is `True` if the value of `<exp>` is a false value and `False` otherwise. (`not` always gives `True` or `False`; `and`/`or` give one of their operands' values.)

The idea

Why these rules? For `and`, a false left side already decides the answer, so Python stops and hands back what it has. For `or`, a true left side already decides it. In both cases the value of the whole expression is *the value of the last operand Python evaluated*. That one sentence answers every such question: find where evaluation stops, and report that operand's value.

Two consequences used on exams. (1) An operand that is skipped has no effect: no printing, no error. `True or 1 / 0` is `True`, with no division by zero. (2) A `print` call as an operand has value `None`, a false value, so `print(1) or 2` continues to 2, and `print(1) and 2` stops with `None`.

```
>>> 1 or 2
1
>>> 3 and 0
0
>>> 0 or '' or None      # all false: value of the last one, None, is not displayed
>>> 2 and 3 and 4
4
>>> print(1) or 2
1
2
>>> print(1) and 2       # displays 1; the value None is not displayed
1
```

Worked exam question (Fall 2022 Midterm 1, Q1(c), from Lecture 5).

```
s = "Knock"
print(print(print(s, s) or print("Who's There?")), "Who?")
```

1. The outer `print` has two operands. The first is `print(`*or-expression*`)`, so the *or-expression* is evaluated first.
2. Its left side `print(s, s)` displays `Knock Knock` and has value `None`. That is false, so the right side is evaluated: displays `Who's There?`, value `None`. The or-expression is `None`.
3. The middle `print(None)` displays `None` and has value `None`.
4. The outer call is `print(None, "Who?")`: displays `None Who?`.

Displayed: `Knock Knock` / `Who's There?` / `None` / `None Who?`.

§2.4 While statements (Lecture 3, CP 1.5)

Execution rule for `while <condition>: <suite>`

1. Evaluate the condition.
2. If it is a true value, execute the *entire* suite, then go back to step 1.
3. If it is a false value, the statement is finished.

- The condition is checked only before each pass. If it becomes false in the middle of the suite, the rest of the suite still runs.
- Some name in the condition must change in the suite, or the loop never ends (unless a `return` inside the loop ends the function).
- Before writing a loop decide: which names are needed, what each one means, and what their starting values are.

Lecture example: prime factorisation. Idea: find the smallest factor $k > 1$ of n (it is prime), print it, divide it out, repeat.

$$858 = 2 \cdot 429 = 2 \cdot 3 \cdot 143 = 2 \cdot 3 \cdot 11 \cdot 13.$$

```
def prime_factors(n):
    """Print the prime factors of n in non-decreasing order."""
    while n > 1:
        k = smallest_factor(n)
        print(k)
        n = n // k

def smallest_factor(n):
    """Return the smallest factor of n greater than 1."""
    k = 2
    while n % k != 0:
        k = k + 1
    return k
```

> **The idea**
>
> The job is split in two: one function finds one factor, the other repeats. Each loop has one name that changes (`k` in one, `n` in the other) and a condition that says when to stop. Why is the smallest factor greater than 1 always prime? If it had a smaller factor greater than 1, that would be a smaller factor of `n`.

The digit loop. Almost every Midterm 1 iteration question walks through the digits of a number from the right:

```
while n > 0:
    d = n % 10      # the current last digit
    ...             # use d
    n = n // 10     # drop it
```

Lecture 4's `same_length(a, b)` counts digits this way (add 1 per pass) for each number and compares the counts.

Worked exam question (Fall 2022 Midterm 1, Q3, from Lecture 5). `classify(n)` returns `'perfect'`, `'abundant'` or `'deficient'` according to whether the sum of the factors of `n` below `n` equals, exceeds, or is less than `n`.

1. What must be computed? The *sum* of the proper factors. So one name accumulates a sum: `total`, starting at `0`.
2. What do we loop over? We cannot loop over factors directly; loop over every integer `k` from `1` to `n - 1` and test each: `n % k == 0`.
3. After the loop, compare `total` with `n`.

```
def classify(n):
    total, k = 0, 1
    while k < n:
        if n % k == 0:
            total = total + k
        k = k + 1
    if total == n:
        return 'perfect'
    elif total < n:
        return 'deficient'
    else:
        return 'abundant'
```

§2.5 Designing functions (Lecture 4, CP 1.4)

- A function's **domain** is the set of arguments it can take; its **range** is the set of values it can return; its **behaviour** is the relationship between input and output. For `square`: any number; a non-negative number; the square of the input.
- Give each function exactly one job, but make it apply to many related situations (`round(1.23)` is `1`, `round(1.23, 1)` is `1.2`).
- **Don't repeat yourself**: implement a process once and call it many times.
- A **docstring** describes the function; examples in it written with `>>>` are **doctests**, run with `python3 -m doctest file.py`.
- A **default argument value**, `def f(x, y=10):`, is used when the call does not supply that argument.

§2.6 Higher-order functions (Lecture 4, CP 1.6)

The question. Summing the first n integers, the first n cubes, and the first n terms of a series for π are the same loop with one line changed. How do we write the loop once?

A **higher-order function** takes a function as an argument or returns a function. This is possible because functions are values (Week 1).

Functions as arguments. The part that changes, "what is the k-th term", becomes a parameter:

```
def cube(k):
    return pow(k, 3)

def summation(n, term):
    """Sum the first n terms of a sequence.
    >>> summation(5, cube)
    225
    """
    total, k = 0, 1
    while k <= n:
        total, k = total + term(k), k + 1
    return total
```

- In `summation(5, cube)`, the operand `cube` is a name; it evaluates to the function. It is *not* called there (no parentheses). The parameter `term` is bound to that function, and `term(k)` calls it.
- `summation(5, cube)` is $1+8+27+64+125 = 225$. With `identity(k)` returning `k`, `summation(5, identity)` is 15. With `pi_term(k)` returning `8 / ((4*k - 3) * (4*k - 1))`, the sums approach π.
- `f` versus `f()`: the first is the function value, the second is the result of calling it.

Functions as return values; nested `def`.

```
def make_adder(n):
    """Return a function that takes one argument k and returns k + n.
    >>> add_three = make_adder(3)
    >>> add_three(4)
    7
    """
    def adder(k):
        return k + n
    return adder
```

- `adder` is defined *while* `make_adder` *is running*, so each call to `make_adder` creates a new `adder` function.
- The body of `adder` uses `n`, which is not one of its own names. It finds `n` in the frame of the `make_adder` call that created it (next subsection).

- `make_adder(3)(4)`: the operator `make_adder(3)` is itself a call expression; it is evaluated first and gives a function, which is then applied to 4.
- `compose1(f, g)` returns the function `h` with `h(x) = f(g(x))`:

```
def compose1(f, g):
    def h(x):
        return f(g(x))
    return h
```

Lecture example: strategies. In the lecture's game, `play(strategy0, strategy1)` takes two functions; each maps the current score to a move. `make_heckling_strategy(strategy, heckle)` returns a new strategy that prints `heckle` and then returns what `strategy` would. A function can be wrapped to add behaviour without changing it.

Program design words from lecture: **modularity** (separate parts with separate jobs), **abstraction** (use a part knowing only what it does, not how), **separation of concerns**.

§2.7 Environments for higher-order functions (Lecture 5)

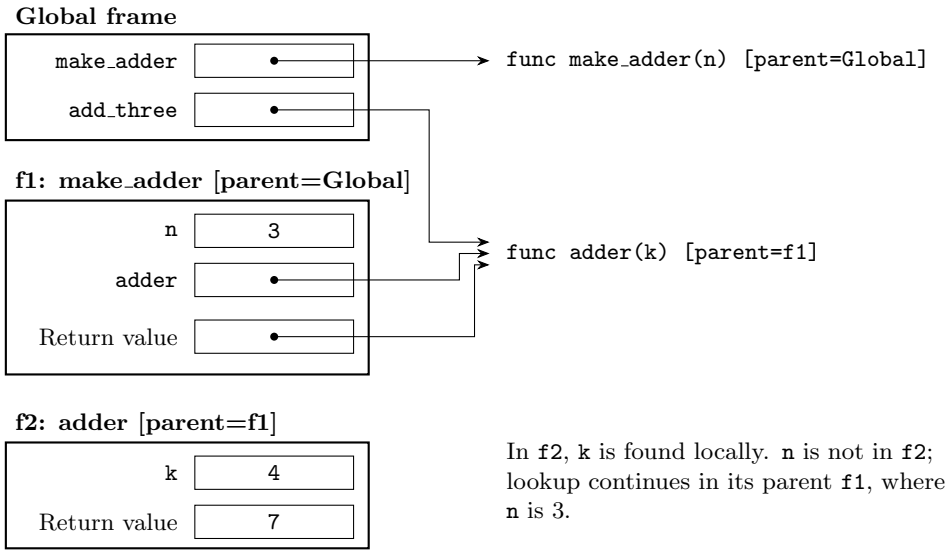
The question. When `add_three(4)` runs, `make_adder(3)` finished long ago. Where does the body `k + n` find `n`?

- **Every function has a parent**: the frame in which its `def` statement was executed.
- **Every local frame has a parent**: the parent of the function that was called.
- The environment of a call is: its local frame, then that frame's parent, then the parent's parent, ..., ending with the global frame. Lookup goes through them in that order.

The idea

The parent is fixed at definition time, not at call time. `adder` was created inside frame `f1` (the call `make_adder(3)`), so its parent is `f1` forever. When it is called later from the global frame, its new frame `f2` has parent `f1`, and lookup of `n` goes `f2` (not there), then `f1` (found: 3). The frame `f1` is kept because a function still refers to it.

So to predict what a name means inside a function, look at where the function was *defined*, never at where it is called.



Environment diagram for `add_three = make_adder(3)` followed by `add_three(4)`.

Additions to the diagram recipe of Week 1.

1. At a `def`: the function's parent is the *current* frame (global, or the local frame being executed).
2. At a call: label the new frame with the function's name from its `def` line and *copy the function's parent* onto the frame.
3. A name bound to a function as an argument works like any binding: with `def twice(f, x): return f(f(x))`, the call `twice(square, 3)` binds `f` to the `square` function in the new frame.

Variations on `make_adder` (the lecture's "wrong make_adder?" quiz). Each variant changes one thing; `print(add_three(4))` follows.

- Add `k = 5` inside `make_adder`, before the inner `def`: prints `7`. The `k` of `adder` is its own parameter in `f2`; the `k` in `f1` is never reached.
- Add `n = 5` inside `make_adder`, before or after the inner `def` but before `return`: prints `9`. The body of `adder` runs later and looks `n` up then; `f1` holds `5` by that time.
- Add `k = 5` as the first line of `adder`: prints `8`. The parameter is rebound in `f2`.
- Add `add_five = make_adder(5)` after `add_three` is created: prints `7`. The second call has its own frame; `f1` is untouched.
- Add `n = 5` in the global frame: prints `7`. Lookup finds `n` in `f1` before reaching global.

Shortcuts

- `and`/`or`: the value is the last operand evaluated. `and` stops at the first false value; `or` stops at the first true value.
- `print(...)` **inside** `and`/`or` is a false value (`None`) that also displays a line.
- **Digit problems**: `while n > 0`, `n % 10`, `n // 10`. Comparing neighbouring digits: `n % 10` and `n // 10 % 10`.
- **Fill-in-the-blank loops**: first say in words what each given name means (`total`, `k`, `count`, `group_size`); the blanks then follow.
- **Accumulating a maximum without** `if`: `largest = max(largest, candidate)`.
- **Function or call?** `f` is a value; `f(x)` runs the body. Count the pairs of parentheses to count the calls.
- **Parents**: write `[parent=...]` on a function the moment it is defined; copy it when the function is called. Never decide a parent from where the call happens.
- **Late lookup**: a name in a function body is looked up when the body runs, so it sees the latest binding in the frame.
- `return` **inside a loop** ends the whole function at once: use it for "find the first ...".

Traps

- `if`/`if` is not `if`/`elif`.
- `0`, `''` and `None` are false values, but they are not equal to `False` in every use: `None == False` is `False`.
- `x == 1 or 2` means `(x == 1) or 2`, which is always a true value.
- A `while` loop whose changing statement is inside an `if` may never end.
- `return print(x)` returns `None`.
- Calling a non-pure function twice to "print then return" doubles its side effects.
- The frame label is the function's defined name (`adder`), not the name it was called by (`add_three`).

§3 Problem set

P1–P6 are original; E1–E2 are real Midterm 1 questions with the published answers in the solutions. Programming exercises are in `exercises/`.

P1 (boolean expressions). What is displayed? Write `ERROR` for an error and *nothing* if nothing is displayed.

```
>>> 0 or 5 and 3
>>> not 0 and None
>>> 1 / 0 or 2
>>> 2 or 1 / 0
>>> print(3) or print(4) and 5
>>> '' or 0
```

(`and` binds more tightly than `or`; `not` more tightly than `and`.)

P2 (if versus elif). What does `f(5)` display with the code as written? What if the second `if` is changed to `elif`?

```
def f(x):
    if x > 3:
        print('a')
        x = x - 3
    if x > 1:
        print('b')
    else:
        print('c')
```

P3 (while). What is displayed?

```
n, steps = 37, 0
while n > 0:
    if n % 2 == 1:
        print(n)
    n, steps = n // 2, steps + 1
print(steps)
```

P4 (write a digit loop by hand). Implement `alternates(n)`, which returns `True` if no two neighbouring digits of the positive integer `n` have the same parity (odd/even), and `False` otherwise. `alternates(2143)` is `True`; `alternates(2241)` is `False`; `alternates(7)` is `True`.

P5 (higher-order functions). What is displayed?

```
def twice(f):
    def g(x):
        return f(f(x))
    return g

def inc(x):
    print('inc', x)
    return x + 1

h = twice(inc)
print(h(0))
print(twice(twice(inc))(10))
```

P6 (environment diagram). Draw the diagram and give what is displayed.

```
x = 10
def outer(x):
    def inner(y):
        return x + y
    x = x * 2
    return inner

f = outer(1)
x = 100
print(f(5))
```

E1 (Fall 2026 Midterm 1, Question 2: New Print, 4 points). Write what is displayed. The code does not error.

```
def new_print(print):
    """Return a better print that returns its argument"""
    def f(x):
        value = print(x)
        if value != None:
            return print('What?')
        return x
    return f

og = print
print = new_print(print)
print = new_print(print)
og('print returned', print(2))
```

E2 (Fall 2026 Midterm 1, Question 3: Long Odds, 4 points). A *group* is a consecutive sequence of digits that are all odd or all even; a group fully inside another is not counted. `num_long_groups(n, k)` returns the number of groups in `n` with at least `k` digits. `15732429999` has groups `1573`, `242`, `9999`: 3 groups with at least 3 digits, 2 with at least 4, 0 with at least 5. You may not write `str`, `len`, `if`, `else`, `[`, `]`, or the digits 3–9 in the blanks.

```
def num_long_groups(n, k):
    group_size, count = 0, 0
    while n:
        group_size += 1
        if ______:          # (a): choose from group_size < k, <= k, > k, >= k,
            count += 1      #      == k, != k, % k == 0, % k != 0
        if ______:          # (b)
            group_size = 0
        n = n // 10
    return count
```